



中国海洋大学
OCEAN UNIVERSITY OF CHINA

第一周实验报告

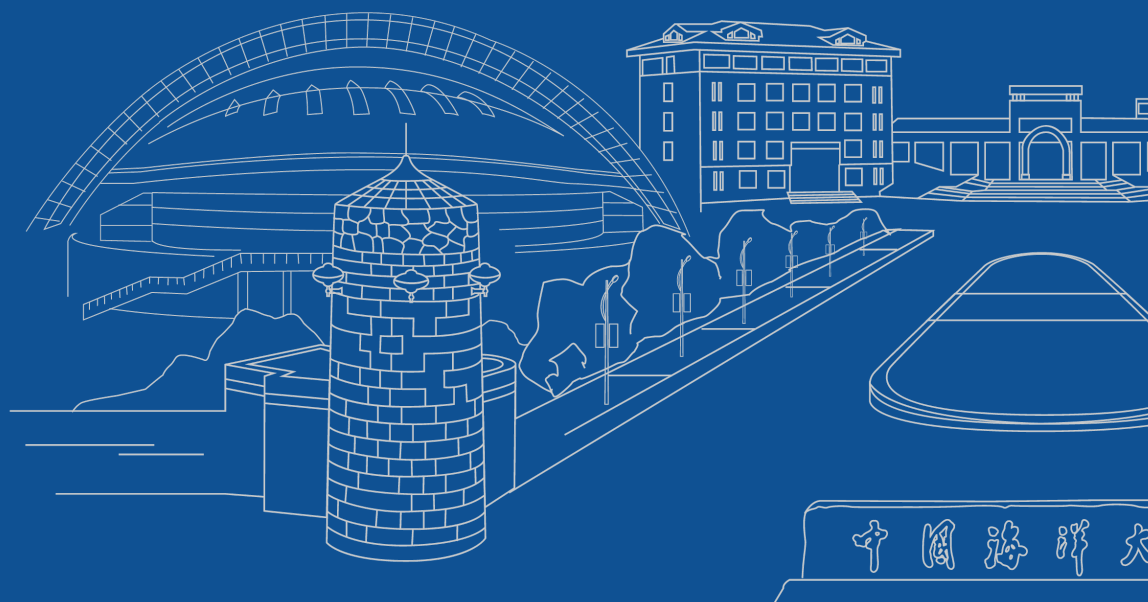
系统开发工具基础

刘晨希

25020007076

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础

2026 年 8 月 31 日





中国海洋大学
OCEAN UNIVERSITY OF CHINA

第一周实验报告

系统开发工具基础

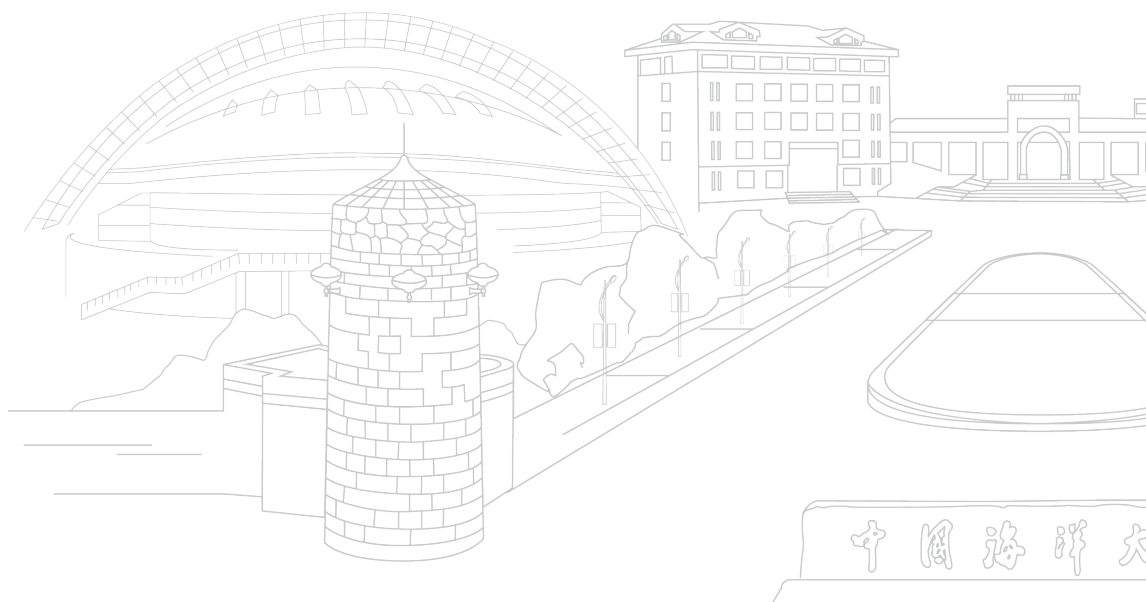
刘晨希

25020007076

指导老师: 周小伟

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础

2026 年 8 月 31 日



目录

目录	i
1 课堂练习及结果	2
1.1 第 1 题含空格文件名的批量整理	2
1.1.1 实验内容	2
1.1.2 实验结果	3
1.2 第 2 题随机访问日志统计	4
1.2.1 实验内容	4
1.2.2 实验结果	5
1.3 第 3 题制造、解决并解释一次合并冲突	6
1.3.1 实验内容	6
1.3.2 实验结果	8
1.4 第 4 题修复并构建一页技术说明	9
1.4.1 实验内容	9
1.4.2 实验结果	11
1.5 Github 仓库地址	12
2 课后练习内容和结果	13
2.1 基础命令与重定向	13
题 1: 确认 Shell 环境	13
题 2: ls -l 的输出格式	13
题 3: glob 模式匹配	14
题 4: 三种引号的区别	15
题 5: 标准流与重定向	15
题 6: 退出状态与 &&/	16
2.2 脚本编写与权限	17
题 7: 为什么 cd 必须是 Shell 内建命令?	17
题 8: 写脚本检查文件是否存在	17
题 9: chmod +x 的作用	18
题 10: set -x 的作用	19
题 11: 带日期的备份文件	20
题 12: 修改 flaky test 脚本接收命令行参数	20
2.3 管道、工具与进阶	22
题 13: 找出 home 目录最常见的 5 种文件扩展名	22
题 14: xargs 结合 find 统计 .sh 文件行数	22

题 15: curl 获取课程网站 HTML, 统计列出了多少讲	23
题 16: jq 处理 JSON	24
题 17: awk 按列过滤并交换列	25
题 18: 拆解 SSH 日志管道 + 从历史记录找出最常用命令	26
3 解题感悟	28

课堂练习及结果

1.1 第 1 题含空格文件名的批量整理

1.1.1 实验内容

首先，如图 1.1 所示，创建 input/docs 和 input/tmp；

```
liuchenxi@liuchenxiMacBook-Air w1 % mkdir q01
liuchenxi@liuchenxiMacBook-Air w1 % cd q01
liuchenxi@liuchenxiMacBook-Air q01 % mkdir input
liuchenxi@liuchenxiMacBook-Air q01 % cd input
liuchenxi@liuchenxiMacBook-Air input % mkdir docs
liuchenxi@liuchenxiMacBook-Air input % mkdir tmp
```

图 1.1: 创建 input/docs 和 input/tmp

如图 1.2 所示，使用 vim 编辑器创建 input/docs/notes one.txt（内容为 alpha 和 beta 两行）、input/docs/.secret.txt（内容为 hidden）、input/tmp/empty.txt（空文件）

```
liuchenxi@liuchenxiMacBook-Air input % cd docs
liuchenxi@liuchenxiMacBook-Air docs % vim "notes one.txt"
liuchenxi@liuchenxiMacBook-Air docs % vim ".secret.txt"
liuchenxi@liuchenxiMacBook-Air docs % cd ../tmp
liuchenxi@liuchenxiMacBook-Air tmp % vim empty.txt
```

图 1.2: 创建 input/docs/notes one.txt、input/docs/.secret.txt、input/tmp/empty.txt

如图 1.3 所示，先使用 vim 编辑器创建空的 run.log，然后使用 date > run.log 和 date » run.log 命令将当前日期分两行写入 run.log，最后使用 cat run.log 命令查看 run.log 的内容。

```
liuchenxi@liuchenxiMacBook-Air input % vim "run.log"
liuchenxi@liuchenxiMacBook-Air input % run.log < date
zsh: no such file or directory: date
liuchenxi@liuchenxiMacBook-Air input % date > run.log
liuchenxi@liuchenxiMacBook-Air input % date » run.log
liuchenxi@liuchenxiMacBook-Air input % cat run.log
2026年 8月 29日 星期六 11时01分22秒 CST
2026年 8月 29日 星期六 11时01分29秒 CST
```

图 1.3: 执行脚本

如图 1.4 所示，使用 pwd 路径显示 q01 的绝对路径。

```
liuchenxi@liuchenxiMacBook-Air input % cd ..
liuchenxi@liuchenxiMacBook-Air q01 % pwd
/Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w1/q01
```

图 1.4: 绝对路径

如图 1.5 所示，先用 find 命令找出 input 目录下的所有项目，然后对于每个找到的项目使用 ls 命令列出其详细信息。

```
liuchenxi@liuchenxiMacBook-Air input % find . -type f -exec ls -la {} \;
-rw-r--r--  1 liuchenxi  staff   96  8月 29 11:01 ./run.log
-rw-r--r--  1 liuchenxi  staff    7  8月 29 10:58 ./docs/.secret.txt
-rw-r--r--  1 liuchenxi  staff   11  8月 29 10:57 ./docs/notes one.txt
-rw-r--r--  1 liuchenxi  staff    0  8月 29 10:59 ./tmp/empty.txt
```

图 1.5: 列出项目

在图 1.5 中, `find` 为查找命令, “.” 表示查找当前目录, `-type f` 表示查找的为文件, `-exec` 表示对每个找到的项目执行后面的命令, 表示当前找到的项目, 表示 `-exec` 命令的结束。而在 `exec` 后的 `ls` 为列出命令, `-l` 为长格式选项, `-a` 为显示所有文件选项, 会显示隐藏文件。

如图 1.6 所示, 利用管道将 `find` 和 `while` 结合起来, 递归遍历当前目录下所有 ‘.txt’ 文件, 在 `work/<25020007076>` 目录里完整复刻原有的文件夹结构, 把全部 `txt` 文件复制过去。

```
liuchenxi@liuchenxiMacBook-Air input % find . -type f -name "*.txt" | while IFS= read -r file; do
  mkdir -p "work/<25020007076>/$(dirname "$file")"
  cp "$file" "work/<25020007076>/$file"
done
```

图 1.6: 复制文件

在图 1.6 中, `IFS` 表示关闭字符串分割, 防止文件名包含空格、制表符被拆分, `read -r file` 表示读取每一行的文件名, `while read -r file; do ... done` 表示对每个读取到的文件名执行 `do` 和 `done` 之间的命令, `mkdir -p` 表示创建目录, `dirname $file` 表示获取文件所在的目录, `cp $file work/<25020007076>/$file` 表示将文件复制到目标目录。

如图 1.7 所示, 将复制后的目录权限设为 750、普通文件权限设为 640; 生成 `inventory.txt`, 列出相对路径和字节数。

```
liuchenxi@liuchenxiMacBook-Air input % find ./work -type d -exec chmod 750 {} +
liuchenxi@liuchenxiMacBook-Air input % find ./work -type f -exec chmod 640 {} +
liuchenxi@liuchenxiMacBook-Air input % find . -type f ! -name "inventory.txt" -print0 | \
while IFS= read -r -d '' file; do
  rel=${file#./}
  size=$(stat -f%z "$file")
  printf "%s\t%s\n" "$rel" "$size"
done > ./inventory.txt
```

图 1.7: 设置权限

在图 1.7 中, `find` 为查找命令, “.” 表示查找当前目录, `-type d` 表示查找的为目录, `-exec` 表示对每个找到的项目执行后面的命令, 表示当前找到的项目, 表示 `-exec` 命令的结束。而在 `exec` 后的 `chmod` 为修改权限命令, 750 表示目录权限为 `rw-r-x-`, 640 表示文件权限为 `rw-r--`。最后使用 `find` 命令查找所有文件并使用 `ls -l` 命令列出详细信息, 并将结果输出到 `inventory.txt` 文件中。

如图 1.8 所示, 使用 `cat` 命令查看 `inventory.txt` 文件的内容。

```
liuchenxi@liuchenxiMacBook-Air input % cat inventory.txt
./run.log 96
./docs/.secret.txt 7
./docs/notes one.txt 11
./work/<25020007076>/docs/.secret.txt 7
./work/<25020007076>/docs/notes one.txt 11
./work/<25020007076>/tmp/empty.txt 0
./tmp/empty.txt 0
```

图 1.8: 查看结果

1.1.2 实验结果

第一题的 github 的 commit 记录和 push 记录分别如图 1.9 所示。


```

liuchenxi@liuchenxiMacBook-Air q02 % vim analyze.sh
liuchenxi@liuchenxiMacBook-Air q02 % cat analyze.sh
#!/bin/bash

# 检查文件是否存在
if [ ! -f "$1" ]; then
    echo "错误: 文件不存在: $1" >&2
    exit 1
fi

echo "HTTP 5xx 数量最多的前 2 个 path: "

awk -F',' 'NR > 1 && $4 >= 500 && $4 < 600 {count[$3]++} END {for (path in count) print path, count[path]}' "$1" |
sort -k2,2nr -k1,1 |
head -n 2

echo
echo "平均 latency_ms: "

awk -F',' 'NR > 1 {sum += $5; count++} END {printf "%.2f\n", sum / count}' "$1"

```

图 1.12: analyze.sh

在图 1.12 中，先判断传入的地址是否存在，如果不存在则将错误写入 stderr 并返回非零退出码。其中 `[ ! -f "$1" ]` 为判断条件。`>&2` 将错误消息输出到标准错误 stderr。在输入“HTTP 5xx 数量最多的前 2 个 path:”的提示后，使用 awk 解析 CSV，跳过表头，统计状态码 5xx 的各 path 出错次数，按计数降序排序后取前 2 条输出。在输入“平均 latency_ms:”的提示后，awk 跳过 CSV 表头，累加延迟字段并统计行数，计算得到保留两位小数的平均延迟并打印。

```

liuchenxi@liuchenxiMacBook-Air q02 % chmod +x analyze.sh

```

图 1.13: 使用 chmod 命令为 analyze.sh 添加执行权限

如图 1.13 所示，使用 chmod 命令为 analyze.sh 添加执行权限。然后如图 1.14 所示，分别对 access.csv 和一个不存在的文件运行脚本 analyze.sh，并查看输出结果。

```

liuchenxi@liuchenxiMacBook-Air q02 % ./analyze.sh access.csv
HTTP 5xx 数量最多的前 2 个 path:
/api/orders 3
/api/users 2

平均 latency_ms:
193.75
liuchenxi@liuchenxiMacBook-Air q02 % ./analyze.sh 111.csv
错误: 文件不存在: 111.csv

```

图 1.14: 执行 analyze.sh

1.2.2 实验结果

第二题的 github 的 commit 记录和 push 记录分别如图 1.15 与图 1.16 所示。

```

liuchenxi@liuchenxiMacBook-Air q02 % git add .
liuchenxi@liuchenxiMacBook-Air q02 % git commit -m "完成了第二个问题02"
[main 9a7464c] 完成了第二个问题02
2 files changed, 27 insertions(+)
create mode 100644 w1/q02/access.csv
create mode 100755 w1/q02/analyze.sh

```

图 1.15: github 记录 2

```

liuchenxi@liuchenxiMacBook-Air lcx_course_work % git push
Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 10 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 935 bytes | 935.00 KiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
3e5aaf7..f0965d3 main -> main

```

图 1.16: github 记录 3

1.3 第3题制造、解决并解释一次合并冲突

1.3.1 实验内容

如图 1.17 所示，使用 git init 命令初始化一个新的 Git 仓库。

```
liuchenxi@liuchenxideMacBook-Air w1 % mkdir q03
liuchenxi@liuchenxideMacBook-Air w1 % cd q03
liuchenxi@liuchenxideMacBook-Air q03 % git init
Initialized empty Git repository in /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w1/q03/.git/
```

图 1.17: 初始化仓库

如图 1.18 所示，创建 config 文件并输入内容 mode=normal。

```
liuchenxi@liuchenxideMacBook-Air q03 % echo "mode=normal" > config.txt
liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
mode=normal
liuchenxi@liuchenxideMacBook-Air q03 % git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    config.txt

nothing added to commit but untracked files present (use "git add" to track)
```

图 1.18: 创建 config

如图 ?? 所示，完成初次提交。

```
liuchenxi@liuchenxideMacBook-Air q03 % echo "mode=normal" > config.txt
liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
mode=normal
liuchenxi@liuchenxideMacBook-Air q03 % git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    config.txt

nothing added to commit but untracked files present (use "git add" to track)
```

图 1.19: 提交 config 文件

如图 1.20 所示，创建 feature-a 分支。

```
liuchenxi@liuchenxideMacBook-Air q03 % git switch -c feature-a
Switched to a new branch 'feature-a'
liuchenxi@liuchenxideMacBook-Air q03 % git branch
* feature-a
main
```

图 1.20: 创建 feature-a 分支

如图 1.21 所示，将 config 文件的内容内容改为 mode=safe 并提交。

```
liuchenxi@liuchenxideMacBook-Air q03 % git switch -c feature-a
Switched to a new branch 'feature-a'
liuchenxi@liuchenxideMacBook-Air q03 % git branch
* feature-a
main
```

图 1.21: 提交 feature-a 分支

如图 1.22 所示，先切换回 main 分支，在 main 分支上创建 feature-b 分支。

```

[liuchenxi@liuchenxideMacBook-Air q03 % git switch main
Switched to branch 'main'
[liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
mode=normal
[liuchenxi@liuchenxideMacBook-Air q03 % git switch -c feature-b
Switched to a new branch 'feature-b'
[liuchenxi@liuchenxideMacBook-Air q03 % git branch
  feature-a
* feature-b
  main

```

图 1.22: 创建 feature-b 分支

如图 1.23 所示，将 config 文件的内容改为 mode=fast 并提交。

```

[liuchenxi@liuchenxideMacBook-Air q03 % echo "mode=fast" > config.txt
[liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
mode=fast
[liuchenxi@liuchenxideMacBook-Air q03 % git add config.txt
[liuchenxi@liuchenxideMacBook-Air q03 % git commit -m "从初始 main 创建 feature-b, 把内容改为 mode=fast 并提交"
[feature-b 2c1309e] 从初始 main 创建 feature-b, 把内容改为 mode=fast 并提交
1 file changed, 1 insertion(+), 1 deletion(-)
[liuchenxi@liuchenxideMacBook-Air q03 % git log --oneline
2c1309e (HEAD -> feature-b) 从初始 main 创建 feature-b, 把内容改为 mode=fast 并提交
b55982b (main) 初始化仓库, 在 main 创建 config.txt, 内容为 mode=normal, 并完成初始提交

```

图 1.23: 提交 feature-b 分支

如图 1.24 所示，切换回 main 分支，尝试将 feature-a 分支合并到 main 分支。从图 1.24 可以看出，没有出现合并错误，成功合并了 feature-a 分支。

```

[liuchenxi@liuchenxideMacBook-Air q03 % git switch main
Switched to branch 'main'
[liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
mode=normal
[liuchenxi@liuchenxideMacBook-Air q03 % git merge feature-a
Updating b55982b..55a4e4d
Fast-forward
 config.txt | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
[liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
mode=safe
[liuchenxi@liuchenxideMacBook-Air q03 % git log --oneline --graph --decorate --all
* 2c1309e (feature-b) 从初始 main 创建 feature-b, 把内容改为 mode=fast 并提交
| * 55a4e4d (HEAD -> main, feature-a) 创建 feature-a, 把内容改为 mode=safe 并提交
|/
* b55982b 初始化仓库, 在 main 创建 config.txt, 内容为 mode=normal, 并完成初始提交

```

图 1.24: 合并 feature-a 分支

从图 1.25 可以看出，尝试将 feature-b 分支合并到 main 分支时，从 git status 的输出中可以看出出现了合并冲突。

```

[liuchenxi@liuchenxideMacBook-Air q03 % git merge feature-b
Auto-merging config.txt
CONFLICT (content): Merge conflict in config.txt
Automatic merge failed; fix conflicts and then commit the result.
[liuchenxi@liuchenxideMacBook-Air q03 % git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   config.txt

no changes added to commit (use "git add" and/or "git commit -a")

```

图 1.25: 合并 feature-b 分支

```

[liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
<<<<<< HEAD
mode=safe
=====
mode=fast
>>>>>> feature-b

```

图 1.26: 查看合并冲突

从图 1.26 可以看出, config 文件的内容出现了冲突, «««< HEAD 表示当前分支的内容, ===== 表示分隔符, »»»> feature-b 表示要合并的分支的内容。

如图 1.27 所示, 解决冲突时保留 mode=safe, 并另加一行 note=reviewed。

```

[liuchenxi@liuchenxideMacBook-Air q03 % printf "mode=safe\nnote=reviewed\n" > config.txt
[liuchenxi@liuchenxideMacBook-Air q03 % cat config.txt
mode=safe
note=reviewed

```

图 1.27: 解决合并冲突

```

[liuchenxi@liuchenxideMacBook-Air q03 % git add config.txt
[liuchenxi@liuchenxideMacBook-Air q03 % git status
On branch main
All conflicts fixed but you are still merging.
(use "git commit" to conclude merge)

Changes to be committed:
  modified:   config.txt

```

图 1.28: 提交解决冲突后的 config 文件

从图 1.28 可以看出, git status 的输出显示已经解决了合并冲突, 并且可以提交解决冲突后的 config 文件。

如图 1.29 所示, 确认工作区干净, 运行 git log -all -graph -decorate -oneline 查看历史。

```

[liuchenxi@liuchenxideMacBook-Air q03 % git commit -m "完成最终提交"
[main 2c5869d] 完成最终提交
[liuchenxi@liuchenxideMacBook-Air q03 % git status
On branch main
nothing to commit, working tree clean
[liuchenxi@liuchenxideMacBook-Air q03 % git log --all --graph --decorate --oneline
* 2c5869d (HEAD -> main) 完成最终提交
| \
| * 2c1309e (feature-b) 从初始 main 创建 feature-b, 把内容改为 mode=fast 并 提交
* | 55a4e4d (feature-a) 创建 feature-a, 把内容改为 mode=safe 并 提交
| /
* b55982b 初始化仓库, 在 main 创建 config.txt, 内容为 mode=normal, 并完成初始提交

```

图 1.29: 查看提交记录

1.3.2 实验结果

如图 1.30 所示, 第三题的 github commit 记录和 push 记录。

```

liuchenxi@liuchenxideMacBook-Air lcx_course_work % git add .
liuchenxi@liuchenxideMacBook-Air lcx_course_work % git commit -m "因第一次完成第三题时未截图，重新做一遍第三题，完成截图操作"
[main e990d40] 因第一次完成第三题时未截图，重新做一遍第三题，完成截图操作
3 files changed, 1 insertion(+)
 create mode 100644 .DS_Store
 create mode 100644 w1/.DS_Store
 create mode 100000 w1/q03_backup
liuchenxi@liuchenxideMacBook-Air lcx_course_work % git push
Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 10 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 1.86 KiB | 1.86 MiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
 ff07d12..e990d40  main -> main

```

图 1.30: github 记录 4

1.4 第 4 题修复并构建一页技术说明

1.4.1 实验内容

如图 1.31 所示，使用 vim 编辑器创建 report.tex 文件，并补全 report.tex 文件的内容。再如图 1.32 所示，使用 cat 命令查看 report.tex 文件的内容。因使用题目要求的 latexmk -pdf -halt-on-error 无法编译中文，所以 report.tex 文件的内容使用英文表示。

```
liuchenxi@liuchenxideMacBook-Air q04 % vim report.tex
```

图 1.31: 创建 report.tex 文件

```

liuchenxi@liuchenxideMacBook-Air q04 % cat report.tex
\documentclass{article}
\usepackage{amsmath}
\begin{document}
\title{Tool Report}
\author{LiuChenxi-25020007076}
\maketitle
\section{Result}
% 在此加入公式、表格及正文引用
\begin{equation}
E = mc^2
\label{eq:example}
\end{equation}

Here,  $E$  represents energy,  $m$  represents mass, and  $c$  represents the speed of light.
The following table is shown in Table~\ref{tab:example}.

\begin{table}[htbp]
\centering
\caption{Example Table}
\label{tab:example}
\begin{tabular}{cc}
\hline
Name & Score \\
\hline
aaa & 90 \\
bbb & 85 \\
\hline
\end{tabular}
\end{table}

According to Equation~\ref{eq:example}, the energy can be calculated. The student scores are shown in Table~\ref{tab:example}.
\end{document}

```

图 1.32: 查看 report.tex 文件

如图 1.33 和图 1.34 所示，使用 latexmk 编译 report.tex 文件，并生成 report.pdf。


```

liuchenxi@liuchenxideMacBook-Air q04 % latexmk -pdf -halt-on-error report.tex
Rc files read (in order):
  NONE
Latexmk: This is Latexmk, John Collins, 9 March 2026. Version 4.88.
No existing .aux file, so I'll make a simple one, and require run of *latex.
[Latexmk: applying rule 'pdflatex'...]
Rule 'pdflatex': Reasons for rerun
[Category 'other':
  Rerun of 'pdflatex' forced or previously required:
  Reason or flag: 'Initial setup'

-----
Run number 1 of rule 'pdflatex'
-----

Running 'pdflatex -halt-on-error -recorder "report.tex"'

-----
This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(.report.tex
LaTeX2e <2026-06-01>
L3 programming layer <2026-08-10>
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/article.cls
Document Class: article 2025/01/22 v1.4n Standard LaTeX document class
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/size10.clo))
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsmath.sty
For additional information on amsmath, use the '?' option.
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amstext.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsgen.sty))
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsbsy.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsopn.sty))
(/usr/local/texlive/2026/texmf-dist/tex/latex/l3kernel/l3backend-pdfTeX.def)
(.report.aux)

LaTeX Warning: Reference `tab:example' on page 1 undefined on input line 15.

LaTeX Warning: Reference `eq:example' on page 1 undefined on input line 31.

LaTeX Warning: Reference `tab:example' on page 1 undefined on input line 31.

[1{/usr/local/texlive/2026/texmf-var/fonts/map/pdfTeX/updmap/pdfTeX.map}]
(.report.aux)

LaTeX Warning: There were undefined references.

LaTeX Warning: Label(s) may have changed. Rerun to get cross-references right.

)</usr/local/texlive/2026/texmf-dist/fonts/type1/public/amsfonts/cm/cmbx10.pfb
></usr/local/texlive/2026/texmf-dist/fonts/type1/public/amsfonts/cm/cmbx12.pfb>
</usr/local/texlive/2026/texmf-dist/fonts/type1/public/amsfonts/cm/cmmi10.pfb><

```

图 1.33: 使用 *latexmk* 编译 *report.tex* 文件

```

=====
Run number 2 of rule 'pdflatex'
=====

Running 'pdflatex -halt-on-error -recorder "report.tex"'

This is pdfTeX, Version 3.141592653-2.6-1.40.29 (TeX Live 2026) (preloaded format=pdflatex)
 restricted \write18 enabled.
entering extended mode
(.report.tex
LaTeX2e <2026-06-01>
L3 programming layer <2026-08-10>
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/article.cls
Document Class: article 2025/01/22 v1.4n Standard LaTeX document class
(/usr/local/texlive/2026/texmf-dist/tex/latex/base/size10.clo))
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsmath.sty
For additional information on amsmath, use the '?' option.
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amstext.sty
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsgen.sty))
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsbsy.sty)
(/usr/local/texlive/2026/texmf-dist/tex/latex/amsmath/amsopn.sty))
(/usr/local/texlive/2026/texmf-dist/tex/latex/l3kernel/l3backend-pdftex.def)
(./report.aux) [1{/usr/local/texlive/2026/texmf-var/fonts/map/pdftex/updmap/pdf
tex.map}] (./report.aux) </usr/local/texlive/2026/texmf-dist/fonts/type1/public
c/amsfonts/cm/cmbx12.pfb></usr/local/texlive/2026/texmf-dist/fonts/type1/public
/amsfonts/cm/cmmi10.pfb></usr/local/texlive/2026/texmf-dist/fonts/type1/public/
amsfonts/cm/cmrr10.pfb></usr/local/texlive/2026/texmf-dist/fonts/type1/public/am
sfonts/cm/cmrr12.pfb></usr/local/texlive/2026/texmf-dist/fonts/type1/public/amsf
onts/cm/cmrr17.pfb></usr/local/texlive/2026/texmf-dist/fonts/type1/public/amsfon
ts/cm/cmrr7.pfb>
Output written on report.pdf (1 page, 65997 bytes).
Transcript written on report.log.
Latexmk: Getting log file 'report.log'
Latexmk: Examining 'report.fls'
Latexmk: Examining 'report.log'
Latexmk: Log file says output to 'report.pdf'
Latexmk: All targets (report.pdf) are up-to-date

```

图 1.34: 使用 latexmk 编译 report.tex 文件

1.4.2 实验结果

如图 1.35 所示，第四题的 github commit 记录和 push 记录。

```

liuchenxi@liuchenxideMacBook-Air q04 % git add q04
fatal: pathspec 'q04' did not match any files
liuchenxi@liuchenxideMacBook-Air q04 % cd ..
liuchenxi@liuchenxideMacBook-Air w1 % git add q04
liuchenxi@liuchenxideMacBook-Air w1 % git commit -m "完成了第四题 q04"
[main ff07d12] 完成了第四题 q04
 6 files changed, 258 insertions(+)
 create mode 100644 w1/q04/report.aux
 create mode 100644 w1/q04/report.fdb_latexmk
 create mode 100644 w1/q04/report.fls
 create mode 100644 w1/q04/report.log
 create mode 100644 w1/q04/report.pdf
 create mode 100644 w1/q04/report.tex
liuchenxi@liuchenxideMacBook-Air w1 % git push
Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 10 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 66.87 KiB | 22.29 MiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:lcx821/lcx_course_work.git
 d058d9d..ff07d12  main -> main

```

图 1.35: github 记录 5

1.5 Github 仓库地址

本次课程的报告、练习、代码等，均已上传至个人 github 仓库。

Github 仓库地址：https://github.com/lcx821/lcx_course_work

其中，q03 练习所用仓库也已上传至 Github。

仓库地址：<https://github.com/lcx821/q03>

课后练习内容和结果

2.1 基础命令与重定向

题 1：确认 Shell 环境

要求：本课程要求使用类 Unix Shell（Bash 或 ZSH）。运行 `echo $SHELL` 确认。

解答：

```
echo $SHELL
```

实际输出：

```
/bin/zsh
```

说明：当前默认 Shell 是 zsh，属于类 Unix Shell，符合课程要求。bash 也可用（版本 3.2.57）。Windows 用户需使用 WSL 或 Linux 虚拟机，不能用 `cmd.exe` 或 PowerShell。

题 2：ls -l 的输出格式

要求：ls -l 的 -l 选项作用是什么？运行 `ls -l /`，每一行最前面的 10 个字符分别代表什么？

解答：

```
ls -l /
```

实际输出（节选）：

```
total 10
drwxrwxr-x 42 root  admin 1344 Aug 30 13:55 Applications
drwxr-xr-x@ 39 root  wheel 1248 Apr  6 16:10 bin
lrwxr-xr-x@  1 root  wheel  11 Apr  6 16:10 etc -> private/etc
```

-l 选项作用：以长格式（long format）列出文件，显示文件类型、权限、硬链接数、所有者、所属组、大小、修改时间和文件名。

前 10 个字符的含义：

例如 `drwxr-xr-x` 表示：这是一个目录，所有者可读写执行，组用户可读执行，其他用户可读执行。

位置	含义	示例
第 1 位	文件类型	d = 目录, l = 符号链接, - = 普通文件
第 2-4 位	所有者 (user) 权限	rwX = 读、写、执行
第 5-7 位	所属组 (group) 权限	r-X = 读、执行 (无写权限)
第 8-10 位	其他用户 (others) 权限	r-X = 读、执行

注：第 11 位可能是 @ (macOS 扩展属性) 或 + (ACL)，不属于权限位。

题 3: glob 模式匹配

要求：什么是 glob? 测试 `ls *.txt`、`ls file?.txt`、`ls \{a,b,c\}.txt` 等模式。
glob 是什么：glob 是 Shell 提供的文件名通配符模式匹配机制。Shell 在执行命令前，会先将包含通配符的参数展开为匹配的文件名列表，再传给程序。

模式	含义
*	匹配任意长度的任意字符 (包括空)
?	匹配恰好一个任意字符
[abc]	匹配 a、b、c 中任意一个字符
[a-z]	匹配 a 到 z 范围内任意一个字符
\{a,b,c\}	大括号扩展，生成 a、b、c 多个字符串

实际测试：

```
mkdir glob_test && cd glob_test
touch file1.txt file2.txt file3.txt a.txt b.txt c.txt abc.txt ab.txt

ls *.txt           # 匹配所有 .txt 文件
ls file?.txt       # 匹配 file + 任意一个字符 + .txt
ls {a,b,c}.txt     # 大括号扩展为 a.txt b.txt c.txt
ls [ab]*.txt       # 匹配以 a 或 b 开头的 .txt 文件
```

输出：

```
=== ls *.txt ===
a.txt ab.txt abc.txt b.txt c.txt file1.txt file2.txt file3.txt

=== ls file?.txt ===
file1.txt file2.txt file3.txt

=== ls {a,b,c}.txt ===
a.txt b.txt c.txt

=== ls [ab]*.txt ===
a.txt ab.txt abc.txt b.txt
```

题 4：三种引号的区别

要求：' 单引号'、" 双引号" 和 \\${ANSI 引号} 有什么区别？写一条命令，输出同时包含字面量 \、! 和换行符的字符串。

引号类型	变量展开	转义序列	历史扩展	适用场景
'...' 单引号	否	否	否	纯字面量字符串
"..." 双引号	是	否（除少数）	可能触发	需要嵌入变量
\\${...} ANSI-C	否	是	否	需要转义序列的字面量

实际测试：

```
# 单引号：完全字面量
echo 'It costs $100! Hello\nWorld'
# 输出：It costs $100! Hello\nWorld

# 双引号：展开变量，\n 不解释
price=100
echo "It costs ${price}! Hello\nWorld"
# 输出：It costs $price! Hello\nWorld

# ANSI-C 引号：解释 \n, $ 是字面量
echo ${'It costs $100! Hello\nWorld'}
# 输出：
# It costs $100! Hello
# World
```

题目要求的命令（同时包含字面量 \、! 和换行符）：

```
echo ${'价格是 $100!\n这是第二行'}
```

输出：

```
价格是 $100!
这是第二行
```

也可以用 `printf '价格是 \${100!\\textbackslash n 这是第二行\\textbackslash n}'`，`printf` 默认就解释转义序列。

题 5：标准流与重定向

要求：Shell 有三条标准流：stdin(0)、stdout(1)、stderr(2)。运行 `ls /nonexistent /tmp`，把 stdout 和 stderr 分别重定向到两个文件。如何把两者都重定向到同一个文件？

三条标准流：

- **stdin (0)**：标准输入，默认来自键盘
- **stdout (1)**：标准输出，默认打印到终端
- **stderr (2)**：标准错误，默认也打印到终端（独立于 stdout）

分别重定向到两个文件：

```
ls /nonexistent /tmp > stdout.txt 2> stderr.txt
```


- `> stdout.txt` 等价于 `1> stdout.txt`，把标准输出写入 `stdout.txt`
- `2> stderr.txt` 把标准错误写入 `stderr.txt`

实际输出：

```
stdout.txt: 包含 /tmp 目录下的文件列表
stderr.txt: 包含 "ls: /nonexistent: No such file or directory"
```

两者重定向到同一个文件（两种方法）：

方法 1（POSIX 标准，推荐）：

```
ls /nonexistent /tmp > both.txt 2>&1
```

- `> both.txt` 先把 `stdout` 指向 `both.txt`
- `2>&1` 把 `stderr` 重定向到当前 `stdout` 指向的地方（即 `both.txt`）
- 顺序很重要：`2>&1` 必须在 `>` 之后

方法 2（bash/zsh 简写）：

```
ls /nonexistent /tmp &> both.txt
```

题 6：退出状态与 `\&\& / ||`

要求：`\$?` 保存上一条命令的退出状态（0=成功）。`\&\&` 仅在前一条成功时执行后一条；`||` 仅在前一条失败时执行后一条。写一行命令：仅当 `/tmp/mydir` 不存在时才创建它。

退出状态：每个命令执行后返回一个 0-255 的整数，0 表示成功，非 0 表示失败。可用 `echo \$?` 查看。

逻辑运算符：

- `cmd1 \&\& cmd2`：cmd1 成功（退出码 0）才执行 cmd2
- `cmd1 || cmd2`：cmd1 失败（退出码非 0）才执行 cmd2
- 两者可组合：`cmd1 \&\& cmd2 || cmd3`（类似三元运算符）

题目要求的命令：

```
[ -d /tmp/mydir ] || mkdir /tmp/mydir
```

解释：

- `[ -d /tmp/mydir ]` 测试目录是否存在（`test -d` 的简写）
- 如果存在，[命令返回 0（成功），|| 后面的 `mkdir` 不执行
- 如果不存在，[返回非 0（失败），|| 触发 `mkdir /tmp/mydir`

实际验证：

```
# 第一次运行：目录不存在，创建成功
[ -d /tmp/mydir ] || mkdir /tmp/mydir
ls -ld /tmp/mydir
# 输出：drwxr-xr-x  2 liuchenxi  wheel  64 Aug 30 16:51 /tmp/mydir

# 第二次运行：目录已存在，不重复创建
[ -d /tmp/mydir ] || mkdir /tmp/mydir
# 无输出，无报错
```

更简洁的等价写法: `mkdir -p /tmp/mydir` (`-p` 表示目录已存在时不报错), 但题目要求用 `\&\&///` 逻辑。

2.2 脚本编写与权限

题 7: 为什么 `cd` 必须是 Shell 内建命令?

要求: 为什么 `cd` 必须是 Shell 内建命令, 而不能是独立程序? (提示: 想想子进程能影响和不能影响父进程的哪些状态。)

核心原因: 子进程无法修改父进程的工作目录。

详细解释:

1. 进程独立性: 在 Unix 系统中, 每个进程都有自己独立的「当前工作目录」(`cwd`)。当 Shell 执行一个外部程序时, 会通过 `fork()` 创建子进程, 再用 `exec()` 加载程序。子进程继承父进程的 `cwd`, 但这只是一个副本。
2. 如果 `cd` 是独立程序:
 - Shell `fork` 出子进程
 - 子进程执行 `cd /some/path`, 修改的是子进程自己的 `cwd`
 - 子进程结束后, Shell (父进程) 的 `cwd` 完全没有改变
 - 结果就是: 你执行了 `cd /some/path`, 但提示符显示的当前目录没变
3. 内建命令的优势: 内建命令由 Shell 自身执行, 不创建子进程, 可以直接修改 Shell 进程自己的 `cwd`, 从而真正改变后续命令的工作目录。

验证:

```
which cd
# 输出: cd: shell built-in command (或 no cd in $PATH)
# 说明 cd 不是独立程序, 而是 Shell 内建命令
```

同类内建命令: `cd`、`export`、`alias`、`exit`、`source/.` 等——凡是需要修改 Shell 自身状态的命令, 都必须是内建命令。

题 8: 写脚本检查文件是否存在

要求: 写一个脚本, 接收文件名参数 (`\$1`), 用 `test -f` 或 `[ -f ... ]` 检查该文件是否存在, 并根据结果输出不同提示。

脚本 `check.sh`:

```
#!/bin/bash
# 检查文件是否存在的脚本

if [ -f "$1" ]; then
    echo "文件 '$1' 存在。"
else
    echo "文件 '$1' 不存在。"
fi
```

关键点：

- \ \$1 是第一个命令行参数
- [-f "\ \$1"] 测试 \ \$1 是否为存在的普通文件 (-f = file)
- "\ \$1" 加双引号防止文件名含空格时出错
- if ...; then ... else ... fi 是 bash 条件语句结构

其他常用测试条件：

条件	含义
-f file	存在且是普通文件
-d file	存在且是目录
-e file	存在（任何类型）
-r file	存在且可读
-w file	存在且可写
-x file	存在且可执行
-s file	存在且大小大于 0

题 9：chmod +x 的作用

要求：把上一题的脚本保存为 check.sh 。先运行 ./check.sh somefile ，会发生什么？然后执行 chmod +x check.sh 再试一次。为什么这一步是必须的？

chmod 前运行：

```
./check.sh somefile
# 输出：bash: ./check.sh: Permission denied
```

chmod 前的权限：

```
-rw----- 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

rw- = 所有者可读、可写，但不可执行

执行 chmod +x check.sh 后：

```
-rwx--x--x 1 liuchenxi staff 147 Aug 30 16:52 check.sh
```

rwx = 所有者可读、可写、可执行

chmod 后运行：

```
./check.sh somefile
# 输出：文件 'somefile' 不存在。

./check.sh check.sh
# 输出：文件 'check.sh' 存在。
```

为什么必须 chmod +x：

- Unix 系统中，文件能否被直接执行取决于可执行权限位（x 位），而不是文件扩展名
- 新建文件默认没有执行权限（安全考虑）
- chmod +x 给所有用户添加执行权限，Shell 才能通过 ./check.sh 直接运行它
- 没有执行权限时，仍可通过解释器显式运行：bash check.sh somefile

shebang 行 (`\#!/bin/bash`) 只在文件有执行权限且被直接运行时才生效。

题 10: `set -x` 的作用

要求：在脚本的 `set` 选项里加入 `-x` 会发生什么？写个简单脚本试试。

`set -x` 的作用：开启调试模式 (xtrace)。Shell 在执行每条命令前，会先把该命令及其展开后的参数打印到 `stderr` (通常以 `+` 开头)，方便追踪脚本执行流程。

演示脚本 `debug\_demo.sh`:

```
#!/bin/bash
set -x

echo "开始执行脚本"
name="Shell"
echo "你好, $name"
for i in 1 2 3; do
    echo "第 $i 次循环"
done
echo "脚本结束"
```

运行输出 (+ 开头的行是 `set -x` 打印的追踪信息):

```
+ echo '开始执行脚本'
开始执行脚本
+ name=Shell
+ echo '你好, Shell'
你好, Shell
+ i=1
+ echo '第 1 次循环'
第 1 次循环
+ i=2
+ echo '第 2 次循环'
第 2 次循环
+ i=3
+ echo '第 3 次循环'
第 3 次循环
+ echo '脚本结束'
脚本结束
```

常用 `set` 选项总结:

选项	作用
<code>set -e</code>	任何命令失败时立即退出脚本
<code>set -u</code>	使用未定义变量时报错 (而非当作空字符串)
<code>set -o pipefail</code>	管道中任一命令失败, 整个管道视为失败
<code>set -x</code>	打印每条执行的命令 (调试用)

推荐组合: `set -euo pipefail` (脚本开头的「严格模式」), 调试时再加 `-x`。

题 11：带日期的备份文件

要求：写一条命令，把文件复制为带当天日期的备份文件名(例如 `notes.txt` → `notes\_2026-01-12.txt`)。提示： `\$(date +%Y-%m-%d)`

命令：

```
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
```

原理：

- `\$(date +%Y-%m-%d)` 是命令替换：执行 `date` 命令，用其输出替换整个 `\$( )`
- `+%Y-%m-%d` 是 `date` 的格式参数：`%Y` = 四位年，`%m` = 两位月，`%d` = 两位日
- 双引号确保日期展开后作为一个整体参数

实际运行：

```
echo "这是笔记内容" > notes.txt
cp notes.txt "notes_$(date +%Y-%m-%d).txt"
ls -l notes*.txt
```

输出：

```
-rw-r--r--  1 staff  19 Aug 30 16:52 notes.txt
-rw-r--r--  1 staff  19 Aug 30 16:52 notes_2026-08-30.txt
```

通用公式：

```
cp "文件名" "文件名_$(date +%Y-%m-%d).扩展名"
```

其他常用日期格式：

```
date +%Y%m%d           # 20260830
date +%Y-%m-%d_%H%M%S  # 2026-08-30_165200 (带时间)
date -v-1d +%Y-%m-%d   # 昨天的日期 (macOS)
```

题 12：修改 flaky test 脚本接收命令行参数

要求：修改讲义中的「复现偶尔才会失败的测试」脚本，使它能够从命令行参数接收测试命令，而不是写死 `cargo test my\_test`。提示：`\$1` 或 `\$@`

修改后的脚本 `flaky\_test.sh`：

```
#!/bin/bash
set -euo pipefail

# 从命令行参数接收测试命令，而不是写死
TEST_CMD="$@"

if [ -z "$TEST_CMD" ]; then
    echo "用法：$0 <测试命令>"
    echo "示例：$0 cargo test my_test"
    exit 1
fi
```

```
# Start CPU stress in background
stress --cpu 8 &
STRESS_PID=$!

# Setup log file
LOGFILE="test_runs_$(date +%s).log"
echo "Logging to $LOGFILE"
echo "运行测试命令: $TEST_CMD"

# Run tests until one fails
RUN=1
while $TEST_CMD > "$LOGFILE" 2>&1; do
    echo "Run $RUN passed"
    ((RUN++))
done

# Cleanup and report
kill $STRESS_PID
echo "Test failed on run $RUN"
echo "Last 20 lines of output:"
tail -n 20 "$LOGFILE"
echo "Full log: $LOGFILE"
```

关键改动:

- 原脚本写死: while cargo test my_test > \"\$LOGFILE\" 2>&1; do
- 改为: TEST_CMD=\"\$@\" 接收所有命令行参数, 然后 while \"\$TEST_CMD\" > ...

特殊参数说明:

参数	含义
\\$1	第一个参数
\\$@	所有参数, 每个参数独立 (加引号后\"\$@\" 保留空格)
\\$*	所有参数, 用 IFS 连接成一个字符串
\\$##	参数个数
\\$0	脚本自身名称
\\$!	最近一个后台进程的 PID

使用方式:

```
./flaky_test.sh cargo test my_test      # 传 cargo test 命令
./flaky_test.sh pytest tests/           # 传 pytest 命令
./flaky_test.sh false                   # 用 false 测试 (第一次就失败)
```

注意: \"\$@\" 比 \"\$1\" 更适合接收完整命令, 因为命令可能包含多个参数 (如 cargo test my_test 是三个参数)。

2.3 管道、工具与进阶

题 13: 找出 home 目录最常见的 5 种文件扩展名

要求: 使用管道找出 home 目录中最常见的 5 种文件扩展名。提示: 组合 find、grep/sed/awk、sort、uniq -c、head

命令:

```
find ~ -type f 2>/dev/null \
| grep -oE '\.[^./]+$' \
| sort \
| uniq -c \
| sort -rn \
| head -n 5
```

逐步拆解:

1. find ~ -type f: 递归列出 home 目录下所有普通文件
2. 2>/dev/null: 把权限错误等 stderr 丢弃
3. grep -oE '\textbackslash .[^{}/.]+\textbackslash \$': 提取每行末尾的扩展名
 - -o: 只输出匹配部分
 - -E: 使用扩展正则
 - \textbackslash .[^{}/.]+\textbackslash \$: 匹配. + 一个或多个非. 非/ 字符 + 行尾
4. sort: 排序 (相同扩展名排在一起)
5. uniq -c: 去重并统计每行出现次数
6. sort -rn: 按数字逆序排序 (出现次数多的在前)
7. head -n 5: 取前 5 个

实际输出:

```
11425 .py
9438 .js
6911 .pyc
5855 .md
3948 .json
```

等价的 awk 版本:

```
find ~ -type f 2>/dev/null | awk -F. '{print "}.${NF}' | sort | uniq -c | sort -rn |
head -5
```

题 14: xargs 结合 find 统计 .sh 文件行数

要求: xargs 把 stdin 的每一行转换为命令参数。结合 find 和 xargs (不要用 find -exec), 找出目录中所有 .sh 文件, 并用 wc -l 统计每个文件行数。加分项: 正确处理文件名中的空格。提示: -print0 和 -0

基础版本:

```
find . -name "*.sh" | xargs wc -l
```

问题：如果文件名包含空格（如 `my script.sh`），`xargs` 会把它拆成两个参数 `my` 和 `script.sh`，导致错误。

正确处理空格的版本（加分项）：

```
find . -name "*.sh" -print0 | xargs -0 wc -l
```

原理：

- `find -print0`：用 `null` 字符（`\0`）分隔文件名，而不是换行符
- `xargs -0`：以 `null` 字符作为分隔符读取输入
- 因为文件名中不可能包含 `null` 字符，所以这种方式 **100%** 安全处理任何文件名（含空格、换行、特殊字符）

实际输出：

```
1 ./dir with spaces/my script.sh
34 ./flaky_test.sh
8 ./check.sh
10 ./debug_demo.sh
53 total
```

`xargs` 常用参数：

参数	含义
<code>-0</code>	以 <code>null</code> 字符分隔输入（配合 <code>find -print0</code> ）
<code>-n N</code>	每次传递 <code>N</code> 个参数给命令
<code>-I \{\}</code>	用 <code>\{\}</code> 作为参数占位符
<code>-P N</code>	并行运行 <code>N</code> 个进程

题 15：curl 获取课程网站 HTML，统计列出了多少讲

要求：使用 `curl` 获取课程网站的 HTML，通过 `grep` 统计列出了多少讲。提示：找出每讲课程名称在 HTML 中的共性；用 `curl -s` 关闭进度输出。

命令：

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/"' \
| sort -u \
| wc -l
```

逐步拆解：

1. `curl -s URL`：静默模式获取网页 HTML（不显示进度条）
2. `grep -oE 'href="/2026/[a-z-]+/'`：提取所有课程链接
 - 课程链接的共性：`href="/2026/课程名/`
 - 课程名由小写字母和连字符组成
3. `sort -u`：排序并去重（`-u = unique`）
4. `wc -l`：统计行数，即课程讲数

实际输出：

```
9
```

列出各讲名称：

```
curl -s https://missing-semester-cn.github.io/2026/ \
| grep -oE 'href="/2026/[a-z-]+/"' \
| sort -u \
| sed 's|href="/2026/||;s|/"|'
```

输出：

```
agentic-coding
beyond-code
code-quality
command-line-environment
course-shell
debugging-profiling
development-environment
shipping-code
version-control
```

共 9 讲，与课程介绍中「九场 1 小时的讲座」一致。

题 16: jq 处理 JSON

要求：用 curl 获取示例数据，再用 jq 提取 version 大于 6 的人员姓名。提示：先 jq . 看结构；再试 jq '.[0] | select(...) | .name'

获取数据并查看结构：

```
curl -s https://microsoftedge.github.io/Demos/json-dummy-data/64KB.json > data.json
jq '.[0]' data.json
```

JSON 结构：

```
{
  "name": "Adeel Solangi",
  "language": "Sindhi",
  "id": "V590F92YF627H FY0",
  "bio": "Donec lobortis eleifend...",
  "version": 6.1
}
```

顶层是一个数组，每个元素包含 name、language、id、bio、version 字段。

提取 version > 6 的人员姓名：

```
jq '.[0] | select(.version > 6) | .name' data.json
```

jq 语法拆解：

- .[]：遍历数组中的每个元素
- select(.version > 6)：筛选 version 字段大于 6 的元素
- .name：提取 name 字段

统计数量：

```
jq '[.[] | select(.version > 6)] | length' data.json
# 输出：89
```

部分输出：

```
"Adeel Solangi"
"Aamir Solangi"
"Adil Eli"
"George Mifsud"
"John Falzon"
...
```

jq 常用操作速查：

表达式	含义
.	整个输入
.field	取字段
.[]	遍历数组
select(cond)	筛选满足条件的元素
length	数组长度或字符串长度
keys	对象的所有键
map(expr)	对数组每个元素应用 expr

题 17：awk 按列过滤并交换列

要求：写一个 awk 命令：只输出第二列大于 100 的行，并交换第一列和第三列。测试数据：printf 'a 50 x\textrightarrow nb 150 y\textrightarrow nc 200 z\textrightarrow n'

命令：

```
printf 'a 50 x\textrightarrow nb 150 y\textrightarrow nc 200 z\textrightarrow n' \
| awk '$2 > 100 { temp=$1; $1=$3; $3=temp; print }'
```

awk 语法拆解：

- `\$2 > 100`：模式（条件），只处理第二列大于 100 的行
- `\{ temp=\$1; \$1=\$3; \$3=temp; print \}`：动作
 - `temp=\$1`：把第一列存到临时变量
 - `\$1=\$3`：第一列替换为第三列的值
 - `\$3=temp`：第三列替换为原来的第一列
 - `print`：打印修改后的整行

实际输出：

```
y 150 b
z 200 c
```

验证：

- 原始行 a 50 x：第二列 50，不大于 100，被过滤
- 原始行 b 150 y：第二列 150 > 100，交换第一列 (b) 和第三列 (y) → y 150 b
- 原始行 c 200 z：第二列 200 > 100，交换第一列 (c) 和第三列 (z) → z 200 c

awk 核心概念：

- awk 按行处理，每行自动按空白字符分成列
- \$1, \$2, \$3... 引用各列，\$0 表示整行
- NR = 当前行号，NF = 当前行列数
- FS = 输入分隔符（默认空白），-F, 可设为逗号
- BEGIN {} 在处理前执行，END {} 在处理后执行

题 18：拆解 SSH 日志管道 + 从历史记录找出最常用命令

要求：拆解讲义中的 SSH 日志处理管道：每一步分别做了什么？然后仿照它构建一个管道，从 ~/.bash_history（或 ~/.zsh_history）中找出最常使用的 Shell 命令。

第一部分：拆解 SSH 日志管道

原始管道：

```
ssh myserver 'journalctl -u sshd -b-1 | grep "Disconnected from" \
| sed -E 's/.*Disconnected from .* user (.*?) [^ ]+ port.*/1/' \
| sort | uniq -c \
| sort -nk1,1 | tail -n10 \
| awk '{print $2}' | paste -sd,
```

逐步拆解：

步骤	命令	作用
1	ssh myserver '...'	登录远程服务器，在远程执行引号内的命令
2	journalctl -u sshd -b-1	查看 sshd 服务在上一次启动时的系统日志
3	grep "Disconnected from"	筛选包含「断开连接」的日志行
4	sed -E 's/...user (.*?) .../1/'	用正则提取用户名，(.*?) 捕获，\ 1 输出
5	sort	对用户名排序（相同名字排在一起）
6	uniq -c	去重并统计每个用户名出现次数
7	sort -nk1,1	按第一列（次数）数值排序（升序）
8	tail -n10	取最后 10 个，即出现次数最多的 10 个
9	awk '{print \$2}'	只输出第二列（用户名），去掉次数
10	paste -sd,	把多行合并为一行，用逗号分隔

最终输出示例：

```
postgres,mysql,oracle,dell,ubuntu,inspur,test,admin,user,root
```

第二部分：从历史记录找出最常用命令

命令 (zsh 用户):

```
LC_ALL=C awk '{print $1}' ~/.zsh_history | sort | uniq -c | sort -rn | head -n 10
```

命令 (bash 用户):

```
awk '{print $1}' ~/.bash_history | sort | uniq -c | sort -rn | head -n 10
```

逐步拆解:

1. `awk '{print $1\}'`: 提取每行的第一个词 (即命令名, 去掉参数)
2. `sort`: 排序
3. `uniq -c`: 统计每个命令出现次数
4. `sort -rn`: 按次数逆序排序
5. `head -n 10`: 取前 10 个

实际输出 (zsh_history):

```
123 brew
25 cd
22 git
20 sudo
14 ls
9 echo
8 fc-match
7 which
7 find
6 mkdir
```

说明:

- zsh 的历史文件默认格式可能包含时间戳前缀 (: 1234567890:0;command), 如果有, 需要先用 `sed 's/\~{}: [0-9]*:[0-9]*;/'` 去掉前缀
- `LC_ALL=C` 防止遇到非 UTF-8 字节时 `awk/sed` 报错
- 最常用的命令是 `brew` (macOS 包管理器), 其次是 `cd`、`git`、`sudo`、`ls`

解题感悟

太难了!!! 对于这一门课程,我是 0 基础,不少同学也都是零基础。即使我认真看完 The Missing Semester of Your CS Education 的讲座视频讲解,并读完讲义后,还是难以完成课堂实验。比如第一步,在终端用命令行创建文件夹,课程中根本没有涉及,我通过大模型的帮助,才知道需要用 `mkdir` 这一个命令来创建文件夹。还有不少 `shell` 命令,单纯我自己是没办法写出来的。不少正则表达式如同课程讲解人 Jon 说的那样,短时间内很难掌握。然后完成任务的过程中,我有时还忘记了截图,比如完成第三题时,我就忘记了截图。写实验报告时,我发现我根本没有第三题的截图,我一度想要通过 Github 上的 `commit` 记录替代终端上的截图,但后面发现这根本没办法满足题目的要求,于是我又重新做了一遍第三题,才将第三题所需要的终端执行窗口截图补齐。

还有 `latex` 的编写,我之前从未接触过 `latex` 的编写,只是听说过 `latex` 对于写论文很有用。老师要求自己设计模板,我觉得这对目前的我来说,在这一周的时间里面很难做到。于是我通过网络搜索到了于景一学长放在 github 上的中国海洋大学实验报告 `latex` 模板,用他的模板完成了实验报告的编写。最初,我准备在 `overleaf` 上编写实验报告,因为在线编写可以省去本地配置环境的麻烦。但是后面我看到要求将报告同步至 Github 仓库,于是我又重新在本地部署了一遍 `latex`。期间遇到不少麻烦,比如有三款字体无法识别。其中两款字体没有在我的电脑上安装,有一款字体没有被正确识别。`vs code` 工作区一度出现了两千多条报错,后面单是为了解决字体的问题,我就耗费了不少的时间。

总体来说,这次的实验无疑是非常困难的,我用了两三天才勉强完成了实验和课堂报告的撰写。困难主要是对 `shell` 和 `latex` 完全不熟悉,但是为 `latex` 配置环境就用了不少时间。但收获还是有的,通过 Jon 的 Youtube 视频,我对 `shell` 命令有了一个了解,第一次知道了 `shell` 的强大。我发现这门课程学习的内容都挺有意思的,无论是对 `shell` 的学习, `latex` 的运用以及 Github 的使用,都是我们未来会接触到的。具有不可替代的现实意义。这是我学习到的内容最实用的一次。通过这次实验,我也基本了解了 `shell` 的使用,并真正直观感受到了 Github 应该怎么用,以及为什么不少人都说 Github 很好用。希望后面的课程我能更加顺利!!!

